Date:24/10/2026

CO2- ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code: DSA0308

Course name: Natural Language Processing

Reg No: 192324315L **Name:** A. Gayathri Shanmukhi

1. Morphological Processing for Search Engine Indexing

Morphological processing is a Natural Language Processing (NLP) technique used to study the structure of words. It identifies the root word and its affixes, including prefixes and suffixes.

There are two main types of morphological transformations:

- Inflectional morphology: Changes the grammatical form of a word without changing its basic meaning or word class. Example: analyze → analyzing.
- Derivational morphology: Forms a related word that may change its meaning or grammatical category. Example: analyze → analytical.

In search engine indexing, morphological normalization helps group related word forms under a common representation. This allows a search engine to retrieve relevant documents even when different forms of related words are used.

Problem Statement

A large-scale search engine processes user queries containing different forms of related words, such as “analyzing”, “analysis”, and “analytical”. These variations may reduce search accuracy if the system treats each word as completely different. To solve this problem, a morphological processing system must identify the root words and affixes, classify the morphological transformations as inflectional or derivational, and normalize the related words into a common representation. A Python program is developed to perform this analysis and generate a structured output for efficient search engine indexing and information retrieval.

Aim

To develop a rule-based morphological processing system to identify roots and affixes, classify word formations, and normalize related words for search engine indexing.

Program

```
words = ["analyzing", "analysis", "analytical"]
rules = {
  "analyzing": ("analyze", "ing", "Inflectional"),
  "analysis": ("analysis", "None", "Derivational"),
  "analytical": ("analysis", "tical", "Derivational")
```

```
}
for word in words:
    root, suffix, category = rules[word]
    normalized = "analysis"
    print("Word:", word)
    print("Root:", root)
    print("Suffix:", suffix)
    print("Type:", category)
    print("Normalized Form:", normalized)
    print()
```

Output

```
====================================
Word: analyzing
Root: analyze
Suffix: ing
Type: Inflectional
Normalized Form: analysis

Word: analysis
Root: analysis
Suffix: None
Type: Derivational
Normalized Form: analysis

Word: analytical
Root: analysis
Suffix: tical
Type: Derivational
Normalized Form: analysis
```

Solution Approach

We solved the problem by developing a rule-based morphological processing program in Python. First, we stored the input words in a list and defined rules to identify their roots, affixes, and morphological categories. Next, we used a loop to process each word and extract the corresponding information from the predefined rules. Finally, we normalized all three words to a common indexing representation, analysis, and displayed the original word, root, affix, transformation type, and normalized form. This approach helps the search engine group related word variations and retrieve relevant documents more efficiently.

Thus, a Python program for morphological processing was implemented successfully. The program identifies the specified word structures and maps all three related words to a common representation, analysis, to support efficient search engine indexing and information retrieval.

3.Morphology-Based Word Normalization Using Python

Morphological processing is an NLP technique used to analyze word structures by identifying roots and affixes. A root carries the basic meaning of a word, while a suffix is added at the end to create a related word.

Derivational morphology forms new words from an existing root. For example, govern is a verb, government is a noun referring to a governing body or system, and governance refers to the process or manner of governing.

In news aggregation, morphological normalization helps group related terms under a common representation. This improves the organization of news articles, topic modeling, and document clustering.

Problem Statement

A text analytics pipeline in a news aggregation system processes words such as “govern”, “government”, and “governance”. These words are related in meaning but have different morphological structures. The system must identify their roots and suffixes, determine their derivational levels, and normalize them to a common representation. A Python program is developed to perform morphological analysis and support topic modeling and document clustering.

Aim

To develop a Python program for morphology-based normalization that identifies the root word and derivational suffixes of “govern”, “government”, and “governance”, and maps them to a common base form for topic modeling and document clustering.

Program

```
words = ["govern", "government", "governance"]

rules = {

    "govern": ("govern", "None", "Verb", 0),

    "government": ("govern", "ment", "Noun", 1),

    "governance": ("govern", "ance", "Noun", 1)

}

print("Morphological Analysis Report")

for word in words:

    root, suffix, category, level = rules[word]

    print("\nWord:", word)

    print("Root:", root)

    print("Suffix:", suffix)
```

```
print("Word Class:", category)
print("Derivational Level:", level)
print("Normalized Form:", root)
```

Output

```
==============================
Morphological Analysis Report

Word: govern
Root: govern
Suffix: None
Word Class: Verb
Derivational Level: 0
Normalized Form: govern

Word: government
Root: govern
Suffix: ment
Word Class: Noun
Derivational Level: 1
Normalized Form: govern

Word: governance
Root: govern
Suffix: ance
Word Class: Noun
Derivational Level: 1
Normalized Form: govern
```

Solution Approach

The problem was solved using a rule-based morphological processing technique in Python. First, the three input words were stored in a list. A dictionary was created to define the root, suffix, and derivational level of each word. The program then processed each word using a loop and retrieved its morphological information. Finally, all three words were normalized to the common representation govern, allowing related terms to be grouped together for topic modeling and document clustering.

Thus, the Python program was implemented successfully to identify the roots, suffixes, and derivational levels of the given words. All three words were normalized to govern, demonstrating how morphological normalization can support efficient news categorization, topic modeling, and document retrieval.